

Fri3ndsDrive

Explicación completa del proyecto: arquitectura, base de datos, inteligencia artificial y Programación Orientada a Objetos

Repositorio	github.com/PEARSTORE-N/Friensdrive
Backend	ASP.NET Core Web API (.NET 10)
Base de datos	Microsoft SQL Server 2025 Developer
Frontend	HTML / CSS / JavaScript (vanilla, sin frameworks)
IA	Google Gemini (gemini-2.5-flash)
Despliegue	Servicio de Windows + Cloudflare Tunnel + Netlify
Autor	Nahum Aquino

Índice

1. Qué es Fri3ndsDrive
2. Arquitectura general del sistema
3. Estructura de carpetas del proyecto
4. El backend, pieza por pieza
 - Program.cs — el punto de arranque
 - Models — las entidades del sistema
 - DTOs — los "sobres" de datos
 - AppDbContext — el puente hacia SQL Server
 - AuthController — registro, login y recuperación
 - ArchivosController — subida, listado, descarga
 - GeminiService — la conexión con la IA
 - TextExtractorService — leer el contenido de los archivos
5. El frontend, pieza por pieza
6. Programación Orientada a Objetos en el proyecto
7. Conexión a la base de datos, en detalle
8. Conexión con la Inteligencia Artificial, en detalle
9. Seguridad aplicada en el proyecto
10. Infraestructura y despliegue (cómo llega esto a internet)
11. Un bug real que apareció y cómo se resolvió
12. Limitaciones actuales y recomendaciones a futuro

1. Qué es Fri3ndsDrive

Fri3ndsDrive es una aplicación web de almacenamiento de archivos personal ("nube" propia) con un plus: cuando subís un archivo, una inteligencia artificial (Google Gemini) lee su contenido, genera automáticamente un **resumen** y un conjunto de **etiquetas** para poder encontrarlo después buscando por tema, no solo por nombre.

El sistema permite: crear una cuenta, iniciar sesión, subir archivos (PDF, Word, Excel, PowerPoint, texto plano, etc.), ver un resumen generado por IA de cada uno, buscar por contenido, descargarlos y eliminarlos. También tiene recuperación de contraseña por código temporal.

En una frase:

Es un sistema de 3 capas clásico — una interfaz web (frontend), una API que contiene toda la lógica de negocio (backend), y una base de datos relacional (SQL Server) — más un servicio externo de IA que se consulta desde el backend.

2. Arquitectura general del sistema

Así viaja una petición desde el navegador del usuario hasta la base de datos y de vuelta:



BASE DE DATOS	SERVICIO EXTERNO
SQL Server (instancia SQLEXPRESS) Base <code>Fri3ndsDriveDB</code>	Google Gemini API (resumen + etiquetas)

Los tres procesos que mantienen esto vivo corren **todo el tiempo** en la misma PC, como Servicios de Windows (arrancan solos, incluso después de un corte de luz):

Servicio de Windows	Qué hace
<code>Fri3ndsDriveApi</code>	Corre la API (.NET) en <code>http://localhost:5126</code>
<code>CloudflaredTunnel</code>	Expone ese puerto local a internet con una URL HTTPS pública
(motor de base de datos)	<code>MSSQL\$SQLEXPRESS</code> — el propio SQL Server

3. Estructura de carpetas del proyecto

```
Fri3ndsDrive/
├── Fri3ndsDrive.Api/           ← Backend (API)
│   ├── Controllers/
│   │   ├── AuthController.cs   Registro, Login, recuperación
│   │   └── ArchivosController.cs Subir, listar, buscar, descargar, eliminar
│   ├── Models/
│   │   ├── Usuario.cs          Entidad de base de datos
│   │   ├── Archivo.cs          Entidad de base de datos
│   │   └── PasswordResetToken.cs Entidad de base de datos
│   ├── DTOs/                   Objetos de transferencia (entrada/salida de la API)
│   ├── Data/
│   │   └── AppDbContext.cs      Configuración de Entity Framework Core
│   ├── Services/
│   │   ├── GeminiService.cs     Conexión con la IA
│   │   └── TextExtractorService.cs Lectura de PDF/Word/Excel/PPTX
│   ├── Uploads/                Archivos físicos subidos por los usuarios
│   ├── Program.cs              Punto de arranque de la aplicación
│   └── appsettings.json         Configuración (sin contraseñas)
├── Fri3ndsDrive.Frontend/      ← Frontend (interfaz web)
│   ├── index.html              Login + dashboard
│   ├── registro.html
│   ├── recuperacion.html
│   ├── app.js                  Toda la lógica del cliente
│   └── styles.css
```

4. El backend, pieza por pieza

4.a — Program.cs: el punto de arranque

Es el primer archivo que se ejecuta. Configura tres cosas fundamentales: qué servicios están disponibles para **inyección de dependencias** (más sobre esto en la sección de POO), cómo se conecta a la base de datos, y qué reglas de CORS aplican (quién tiene permiso de llamar a la API desde un navegador).

```
var builder = WebApplication.CreateBuilder(args);

builder.Host.UseWindowsService(); // permite correr como servicio de Windows

builder.Services.AddControllers();
builder.Services.AddHttpClient<GeminiService>(); // GeminiService puede hacer llamadas HTTP
builder.Services.AddScoped<TextExtractorService>();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddCors(options => {
    options.AddPolicy("Fri3ndsDrivePolicy", policy => {
        policy.WithOrigins("https://fri3ndsdrive.netlify.app")
            .AllowAnyHeader().AllowAnyMethod();
    });
});
```

Notá algo importante: la cadena de conexión (`GetConnectionString("DefaultConnection")`) no tiene la contraseña escrita ahí — la levanta de una variable de entorno del sistema operativo en tiempo de ejecución (explicado en la sección 6).

4.b — Models: las entidades del sistema

Cada clase en `Models/` representa una tabla de la base de datos. Son las piezas centrales del diseño orientado a objetos del proyecto.

Models/Usuario.cs

```
public class Usuario
{
    [Key] public int IdUsuario { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Correo { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public List<Archivo> Archivos { get; set; } = new();
}
```

Un usuario **tiene muchos** archivos — de ahí la lista `Archivos`. Nunca se guarda la contraseña real, solo su `PasswordHash` (ver sección 8).

Models/Archivo.cs

```
public class Archivo
{
    [Key] public int IdArchivo { get; set; }
    public int IdUsuario { get; set; }
    public string NombreOriginal { get; set; }
    public string RutaArchivo { get; set; }
    public string? TextoExtraido { get; set; }
    public string? ResumenIA { get; set; }
    public string? EtiquetasIA { get; set; }

    [ForeignKey("IdUsuario")]
    public Usuario? Usuario { get; set; }
}
```

Cada archivo **pertenece a** un usuario (relación inversa a la de arriba). Guarda la ruta física en disco (`RutaArchivo`) y los tres campos que produce la IA: el texto que se le extrajo, el resumen y las etiquetas.

4.c — DTOs: los "sobres" de datos

Un **DTO** (Data Transfer Object) es una clase que solo transporta datos entre el frontend y el backend — no tiene lógica ni representa una tabla. Se usan para no exponer directamente los `Models` por la API: así el cliente solo ve/manda exactamente los campos que necesita, ni uno más.

```
public class RegistroDTO
{
    public string Nombre { get; set; }
    public string Correo { get; set; }
    public string Password { get; set; }
}
```

¿Por qué no usar directamente el modelo `Usuario` como entrada del registro?

Porque `Usuario` tiene campos como `PasswordHash` , `IdUsuario` o `Activo` que el cliente no debería poder mandar ni ver — el DTO actúa como un filtro de seguridad y de forma.

4.d — ApplicationDbContext: el puente hacia SQL Server

Esta clase es el corazón de **Entity Framework Core**, la tecnología que traduce clases de C# en filas de SQL Server sin que el programador tenga que escribir sentencias SQL a mano. Hereda de `DbContext` (una clase que provee Microsoft) — es el ejemplo más claro de **herencia** en el proyecto.

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Usuario> Usuarios { get; set; }
    public DbSet<Archivo> Archivos { get; set; }
    public DbSet<PasswordResetToken> PasswordResetTokens { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Usuario>(entity => {
            entity.HasIndex(u => u.Correo).IsUnique(); // no puede haber dos correos iguales
        });

        modelBuilder.Entity<Archivo>(entity => {
            entity.HasOne(a => a.Usuario)
                .WithMany(u => u.Archivos)
                .HasForeignKey(a => a.IdUsuario); // clave foránea Archivo → Usuario
        });
    }
}
```

Cada `DbSet<T>` es, en la práctica, "la tabla T" — `context.Usuarios.Add(...)` es literalmente un `INSERT`, y EF Core arma el SQL real por detrás. El método `OnModelCreating` **sobrescribe** (`override`) el comportamiento de la clase base para definir relaciones e índices — otro ejemplo de herencia/polimorfismo.

4.e — AuthController: registro, login y recuperación

Expone cuatro endpoints bajo la ruta `/api/auth`:

Endpoint	Qué hace
<code>POST /api/auth/registro</code>	Crea un usuario nuevo. Verifica que el correo no exista, hashea la contraseña con <code>BCrypt</code> y la guarda.
<code>POST /api/auth/login</code>	Busca el usuario por correo y compara la contraseña con <code>BCrypt.Verify(...)</code> .
<code>POST /api/auth/solicitar-recuperacion</code>	Genera un código de 6 dígitos con vencimiento de 10 minutos y lo guarda en <code>PasswordResetTokens</code> .
<code>POST /api/auth/restablecer-password</code>	Valida el código (que no esté vencido ni usado) y actualiza el hash de la contraseña.

4.f — ArchivosController: subida, listado, búsqueda, descarga

Endpoint	Qué hace
<code>POST /api/archivos/subir</code>	Recibe el archivo, lo guarda en disco, le extrae el texto, llama a la IA para resumen + etiquetas, y guarda todo en la base.

GET /api/archivos/usuario/{id}	Lista todos los archivos de un usuario.
GET /api/archivos/buscar	Busca por nombre, tipo, resumen, texto extraído o etiquetas (usando Contains de LINQ).
GET /api/archivos/descargar/{id}	Devuelve el archivo físico desde disco.
DELETE /api/archivos/eliminar/{id}	Borra el archivo físico y su registro en la base.

4.g — GeminiService: la conexión con la IA

Ver sección 7 para el detalle completo. En resumen: recibe el texto extraído del archivo y le pide a la IA de Google (Gemini) que genere un resumen y, en otra llamada, etiquetas de clasificación.

4.h — TextExtractorService: leer el contenido de los archivos

Antes de poder resumir un archivo, hay que sacarle el texto — y cada formato se lee distinto. Este servicio usa un **switch** sobre la extensión del archivo para delegar a un método distinto según el tipo:

```
public string ExtraerTexto(string ruta, string nombreOriginal)
{
    string extension = Path.GetExtension(nombreOriginal).ToLower();
    return extension switch
    {
        ".txt" => LeerTextoPlano(ruta),
        ".pdf" => LeerPdf(ruta),           // Librería PdfPig
        ".docx" => LeerWord(ruta),         // Librería DocumentFormat.OpenXml
        ".xlsx" => LeerExcel(ruta),        // Librería ExcelDataReader
        ".pptx" => LeerPowerPoint(ruta),
        _ => ""
    };
}
```

Si el tipo de archivo no tiene texto legible (una imagen, por ejemplo), devuelve texto vacío y el sistema avisa: "Este tipo de archivo no contiene texto legible o requiere OCR."

5. El frontend, pieza por pieza

El frontend es HTML + CSS + JavaScript **puro**, sin frameworks (no usa React, Vue ni Angular). Son 3 páginas HTML (login/dashboard, registro, recuperación) y un único archivo **app.js** con toda la lógica.

Archivo	Rol
index.html	Pantalla de login y, una vez logueado, el dashboard con la lista de archivos.
registro.html	Formulario de alta de cuenta.

<code>recuperacion.html</code>	Solicitud y confirmación del código de recuperación.
<code>app.js</code>	Todas las funciones: <code>iniciarSesion()</code> , <code>subirArchivo()</code> , <code>cargarArchivos()</code> , <code>buscarArchivos()</code> , <code>descargarArchivo()</code> , <code>eliminarArchivo()</code> , etc.

Toda la comunicación con el backend pasa por `fetch()` hacia la constante `API_URL` (la URL pública del túnel de Cloudflare). Por ejemplo, así se sube un archivo:

```
const formData = new FormData();
formData.append("archivo", archivo);
formData.append("idUsuario", usuarioActual.idUsuario);

const respuesta = await fetch(`${API_URL}/archivos/subir`, {
  method: "POST",
  body: formData
});
```

Detalle importante sobre la sesión:

La variable `usuarioActual` vive solo en memoria del navegador (una variable de JavaScript). Si recargás la página, se pierde la sesión y hay que volver a loguearse — no hay tokens persistentes (JWT) ni cookies de sesión. Funciona bien para un proyecto personal/académico, pero es una simplificación a tener en cuenta.

6. Programación Orientada a Objetos en el proyecto

El backend está escrito en C#, un lenguaje orientado a objetos por diseño. Estos son los pilares de POO identificables en el código real del proyecto:

① Clases y objetos

Todo el backend está organizado en clases: `Usuario`, `Archivo`, `PasswordResetToken` (entidades), `AuthController`, `ArchivosController` (controladores), `GeminiService`, `TextExtractorService` (servicios), y los DTOs. Cada vez que llega una petición HTTP, ASP.NET Core crea una **instancia** (un objeto) del controlador correspondiente para atenderla.

② Encapsulación

Las propiedades de los modelos (`public int IdUsuario { get; set; }`) usan *auto-propiedades* de C#, que controlan el acceso a los datos internos del objeto a través de `get / set` en vez de exponer campos directamente. Un ejemplo más estricto de encapsulación es que **la contraseña real del usuario nunca se guarda ni se expone** — solo existe temporalmente en memoria durante el registro/login, y lo único que persiste es `PasswordHash`, un dato derivado e irreversible.

③ Herencia

`AppDbContext : DbContext` hereda toda la funcionalidad de Entity Framework Core (seguimiento de cambios, generación de SQL, transacciones) y la extiende con las tablas propias del proyecto. De la misma manera, cada controlador hereda de `ControllerBase`: `public class AuthController : ControllerBase`, lo que le da automáticamente métodos como `Ok(...)`, `BadRequest(...)` o `NotFound(...)` para construir respuestas HTTP.

④ Polimorfismo

El método `protected override void OnModelCreating(...)` en `AppDbContext` **sobrescribe** el comportamiento por defecto de la clase base `DbContext` — es polimorfismo de sobrescritura (override). También ocurre implícitamente cada vez que un controlador devuelve `ActionResult`: el tipo real devuelto (`OkObjectResult`, `BadRequestObjectResult`, `NotFoundObjectResult` ...) varía según el caso, pero todos se tratan de forma uniforme a través de esa interfaz común.

⑤ Abstracción e interfaces

El código depende de **abstracciones**, no de implementaciones concretas: `IFormFile` (un archivo subido, sin importar cómo llegó por HTTP), `IWebHostEnvironment` (información del entorno de hosting, usada para encontrar la carpeta correcta de `Uploads`) e `IConfiguration` (acceso a configuración/variables de entorno, usado en `GeminiService` para leer la API key). El código de negocio nunca sabe ni le importa la clase concreta detrás de esas interfaces.

⑥ Composición e inyección de dependencias

En vez de que cada clase cree sus propias dependencias con `new`, ASP.NET Core se las "inyecta" por el constructor — esto es **Inyección de Dependencias** (un patrón de diseño basado en composición de objetos):

```
public class ArchivosController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly GeminiService _geminiService;
    private readonly TextExtractorService _textExtractorService;

    public ArchivosController(AppDbContext context, IWebHostEnvironment env,
                             GeminiService geminiService, TextExtractorService
                             textExtractorService)
    {
        _context = context;
        _geminiService = geminiService;
        _textExtractorService = textExtractorService;
    }
}
```

Esto también se ve a nivel de datos: un `Usuario` tiene una lista de objetos `Archivo` (`List<Archivo> Archivos`) — el objeto `Usuario` está compuesto, en parte, por sus archivos.

7. Conexión a la base de datos, en detalle

La conexión ocurre en tres niveles:

1. **Motor:** SQL Server 2025 Developer, instancia con nombre `SQLEXPRESS`, con protocolo TCP/IP y autenticación mixta (Windows + SQL) habilitados.
2. **Usuario de aplicación:** un login de SQL dedicado, `fri3ndsdrive`, creado específicamente para esta app — *no* es una cuenta administradora. Solo tiene los roles `db_datareader` y `db_datawriter` sobre la base `Fri3ndsDriveDB`, es decir: puede leer y escribir datos, pero no puede tocar otras bases, crear/borrar tablas ni administrar el servidor.
3. **Cadena de conexión:** en vez de estar escrita en `appsettings.json` (donde quedaría expuesta si el código se sube a GitHub), vive como una variable de entorno del sistema operativo:

Nombre de la variable (nivel Machine):
`ConnectionStrings__DefaultConnection`

Valor:
`Server=localhost\SQLEXPRESS;Database=Fri3ndsDriveDB;
User Id=fri3ndsdrive;Password=***;TrustServerCertificate=True;`

¿Por qué funciona esto sin cambiar código?

ASP.NET Core lee la configuración en capas: primero `appsettings.json`, y después las variables de entorno —que

sobrescriben

lo anterior si tienen el mismo nombre (usando `__` como separador de niveles, equivalente a `ConnectionStrings:DefaultConnection` en el JSON). Por eso alcanza con setear la variable de entorno: la app la toma automáticamente sin tocar una línea de `Program.cs`.

Entity Framework Core (`AppDbContext`) traduce las operaciones de C# a SQL real. Por ejemplo, `_context.Usuarios.Add(usuario); await _context.SaveChangesAsync();` genera, por detrás, un `INSERT INTO Usuarios (...) VALUES (...)`.

8. Conexión con la Inteligencia Artificial, en detalle

La IA se usa en dos momentos, ambos dentro de `GeminiService.cs`, justo después de subir un archivo:

Método	Qué le pide a la IA
<code>GenerarResumenAsync(contenido)</code>	"Resume el siguiente documento de forma breve, profesional y clara"
<code>GenerarEtiquetasAsync(contenido)</code>	"Genera únicamente entre 3 y 6 etiquetas separadas por comas para clasificar este documento"

El flujo técnico es una llamada HTTP directa (sin SDK) al endpoint REST de Gemini:

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent
Header: x-goog-api-key: <API KEY>
Body: { "contents": [{ "parts": [{ "text": "Resume el siguiente documento..." }] }] }
```

El servicio arma el `HttpClient` (inyectado automáticamente vía `builder.Services.AddHttpClient<GeminiService>()` en `Program.cs`), agrega el header `x-goog-api-key`, serializa el pedido a JSON, y al recibir la respuesta navega el JSON hasta llegar al texto generado:

```
document.RootElement
  .GetProperty("candidates")[0]
  .GetProperty("content")
  .GetProperty("parts")[0]
  .GetProperty("text")
  .GetString();
```

¿De dónde sale el texto que se le manda a la IA?

No se le manda el archivo en sí — se le manda el **texto ya extraído**

por `TextExtractorService` (sección 4.h), recortado a un máximo de 6000 caracteres para no exceder límites del modelo. Si el archivo no tiene texto legible (por ejemplo, una imagen), directamente no se llama a la IA: se guarda un mensaje fijo.

Dónde vive la API key:

Igual que la contraseña de la base de datos, la API key de Gemini

no está en el código ni en `appsettings.json`

. Vive como variable de entorno del sistema (`Gemini__ApiKey`), separada del repositorio de GitHub.

9. Seguridad aplicada en el proyecto

Aspecto	Cómo se resolvió
Contraseñas de usuarios	Hasheadas con BCrypt (algoritmo lento a propósito, resistente a fuerza bruta). Nunca se guarda ni se compara la contraseña en texto plano.
Secretos (API key, contraseña de BD)	Fuera del código, como variables de entorno del sistema operativo — nunca llegan al repositorio de GitHub.
Usuario de base de datos	Cuenta <code>fri3ndsdrive</code> de mínimo privilegio: solo lectura/escritura sobre una base, no es administradora del servidor SQL.
CORS	Restringido a un único origen permitido (<code>https://fri3ndsdrive.netlify.app</code>) en vez de aceptar cualquier sitio web.
Transporte	Todo el tráfico público pasa por HTTPS gracias al certificado que provee automáticamente el túnel de Cloudflare.
Repositorio	Privado en GitHub, con <code>.gitignore</code> excluyendo <code>bin/</code> , <code>obj/</code> , <code>Uploads/</code> y configuración local.

10. Infraestructura y despliegue (cómo llega esto a internet)

La PC de Nahum actúa como servidor propio ("self-hosting"). No hay ningún proveedor de nube de por medio para el backend ni la base de datos — todo corre en el hardware local, expuesto a internet mediante un túnel.

Los tres servicios de Windows que sostienen el sistema

Servicio	Comando de creación	Función
Fri3ndsDriveApi	<code>sc.exe create Fri3ndsDriveApi binPath="C:\Fri3ndsDrive\api\Fri3ndsDrive.Api.exe" start=auto</code>	Corre la API .NET, reinicia solo si se cae, arranca con Windows.
CloudflaredTunnel	<code>sc.exe create CloudflaredTunnel binPath="cloudflared.exe tunnel --url http://localhost:5126" start= auto</code>	Publica la API con una URL HTTPS pública sin abrir puertos en el router.
MSSQL\$SQLEXPRESS	(creado por el instalador de SQL Server)	El propio motor de base de datos.

Flujo de una petición real (ej: subir un archivo)



Limitación conocida — la URL pública no es fija:

Como no hay un dominio propio registrado, se usa un "Quick Tunnel" de Cloudflare (gratuito, sin cuenta obligatoria para ese modo) — pero

cada vez que el servicio `CloudflaredTunnel` se reinicia

(por ejemplo, tras reiniciar la PC), Cloudflare asigna una URL nueva al azar. Hay que leerla en

`C:\Fri3ndsDrive\tunnel.log` y actualizar la constante `API_URL` en `app.js`, volviendo a desplegar el frontend en Netlify. Si en el futuro se compra un dominio propio y se agrega a Cloudflare, se puede crear un

túnel con nombre

que tiene una URL fija para siempre.

11. Un bug real que apareció y cómo se resolvió

Durante las pruebas post-despliegue, los archivos subidos no aparecían donde se esperaba. La causa: cuando un proceso .NET corre como **Servicio de Windows** (a diferencia de correrlo manualmente con `dotnet run`), el Administrador de Control de Servicios (SCM) lo arranca con el directorio de trabajo en `C:\Windows\System32`, no en la carpeta de la aplicación.

El código original construía la ruta así:

```
string carpetaUploads = Path.Combine(Directory.GetCurrentDirectory(), "Uploads");  
// Directory.GetCurrentDirectory() devolvía C:\Windows\System32 al correr como servicio
```

La corrección fue usar la ruta real de la aplicación en vez de "la carpeta actual del proceso":

```
string carpetaUploads = Path.Combine(_env.ContentRootPath, "Uploads");  
// _env es IWebHostEnvironment — siempre apunta a la carpeta de la app, sin importar cómo se ejecute
```

Lección:

nunca confiar en `Directory.GetCurrentDirectory()` dentro de una app que puede correr de formas distintas (consola, servicio, contenedor). `IWebHostEnvironment.ContentRootPath` es la forma correcta y portable de referirse a "la carpeta de mi aplicación" en ASP.NET Core.

12. Limitaciones actuales y recomendaciones a futuro

Limitación actual	Posible mejora futura
La sesión del usuario vive solo en una variable de JavaScript (se pierde al recargar la página).	Implementar autenticación con tokens JWT guardados en <code>localStorage</code> o cookies seguras.

El código de recuperación de contraseña se devuelve en la respuesta de la API (para poder probarlo sin correo configurado).	Enviarlo por email real (ej. con SendGrid o SMTP) y dejar de exponerlo en la respuesta.
La URL pública del backend cambia con cada reinicio del túnel.	Comprar un dominio propio y crear un túnel de Cloudflare con nombre fijo.
Los archivos se guardan en el disco local de una sola PC, sin backup automático.	Backups periódicos, o migrar el almacenamiento a algo tipo Azure Blob Storage / S3.
Si la PC se apaga, todo el sistema deja de estar disponible (no hay alta disponibilidad).	Aceptable para un proyecto personal; para producción real convendría un VPS o un servicio cloud administrado.

Documento generado como parte de la documentación técnica del proyecto Fri3ndsDrive.